



TECHNISCHE
UNIVERSITÄT
WIEN

ottobock.

Framework for System Simulation

Thesis – FEM Integration using Netgen/NGSolve

Florian Frech, 29.09.2025

1. Framework Architecture Overview

2. Demo Model: Controlled Pendulum with Wall Impact

3. Theoretical Background (Jupyter Notebook)

- Material Laws: Linear elastic and Hyperelastic
- Time integration with Newmark scheme
- Contact mechanics within NGSolve

4. Validation Example: Ball-Wall Contact (Jupyter Notebook)

- Contact definition & Example Gap Calculation
- Absolute vs. Incremental Gap Function
- Validation by Energy Conservation

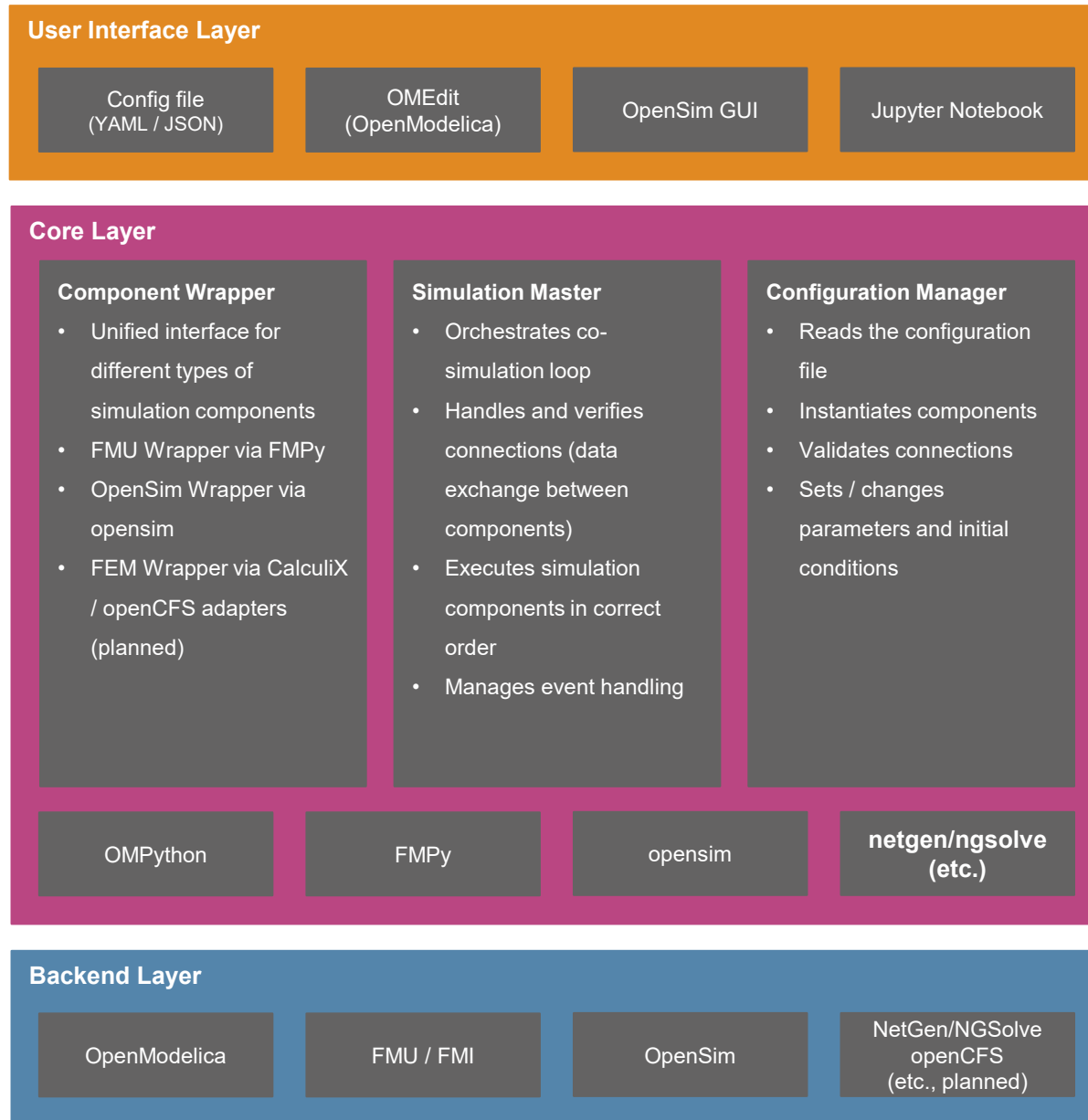
5. FEM Pendulum Model (Jupyter Notebook)

- Configuration
- Geometry and Mesh
- FE Spaces and Variational Formulation
- Contact Definition
- Applying Initial Conditions
- Simulation Loop
- Visualizations

6. Integration into the Framework (Jupyter Notebook)

- Modularization of FEM Model functionality
- Implementation of Co-Simulation Component Methods
- Orchestration in Master-Simulation Loop
- Synchronization & Execution Switch OpenSim / FEM

7. Next Steps



User Interface Layer:

- Uses OMEdit for equation-based model definition (Modelica)
- Uses OpenSim GUI for visualization of OpenSim Model
- Configuration files:
 - File paths to FMUs, OpenSim models, FEM models
 - Defining values for parameters and initial values
 - Simulation settings: start/stop time, step size, solver settings
 - Logging and output options
- Jupyter Notebook: used as interactive orchestration environment

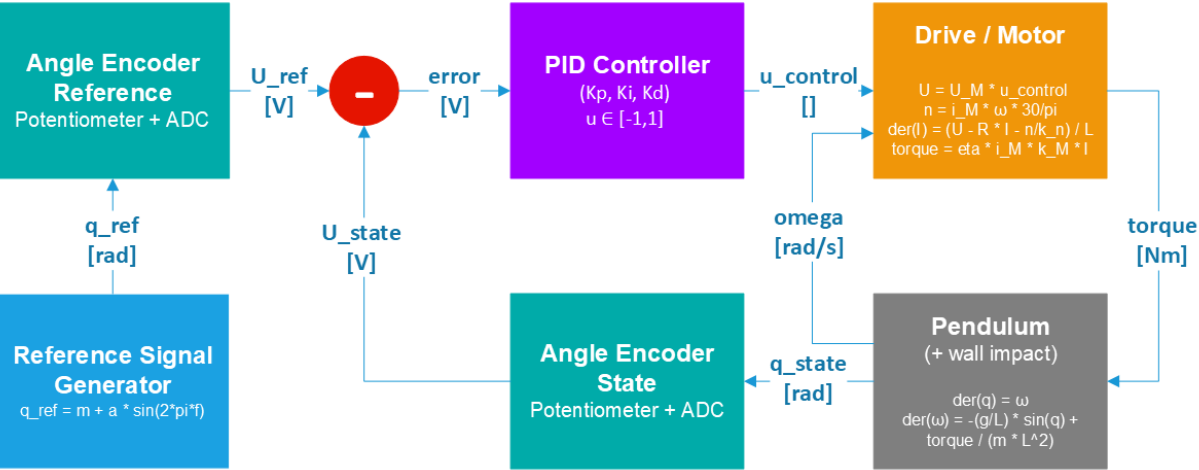
Core Layer:

- Core layer between the user definitions and the solver / model backends
- Provides a unified interface protocol for simulation components
- Simulation master orchestrates the execution of the subsystem models
- Configuration file is parsed, and corresponding system model is set up, connections are validated

Backend Layer:

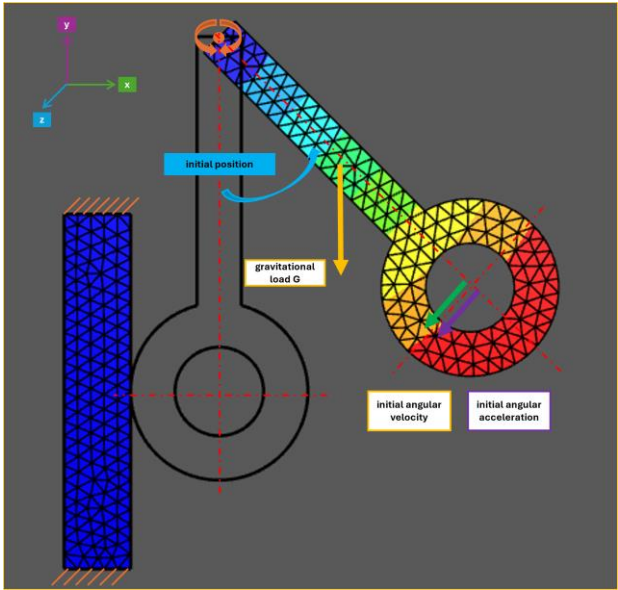
- Layer represents the heterogeneous simulation engines
- Each backend is accessed exclusively through a wrapper, ensuring unified protocol compliance
- Equation-based models are built in OpenModelica and exports as Co-Simulation FMUs via OMPython
- FMUs are imported and simulated via FMPy (solver is inside the FMU)
- OpenSim models are simulated using opensim Python API
- **FEM model integration via Netgen/NGSolve**

Simplified Block Diagram:



Suitability as Framework Demonstrator:

- **Simple benchmark** system with mechanical, electrical, and control subsystems
- **Captures core challenges of system simulation:**
 - Continuous + discrete dynamics
 - Sensors + quantization
 - Feedback control
 - Heterogeneous model integration
- **Provides a progression of complexity:**
 1. Equation-based pendulum + controller (FMUs)
 2. Hybrid extension (Impact Wall = discrete event)
 3. Replacement Pendulum with OpenSim model
 4. Integration with FEM (elastic rod/contact with wall)
- Serves **as proof-of-concept for framework capabilities**



Subsystem	Description
Reference Signal Generator	• Creates a sine signal oscillating around a mean with defined amplitude and frequency • Provides the reference input for the control loop (ideal trajectory)
Angle Encoder	• Maps the pendulum's angular position into an analog voltage using a potentiometer model • Adds a quantization via ADC with $nBits$ resolution and clips to ADC voltage range
PID Controller	• Compares the reference signal (U_ref) and state signal (U_state) to compute an error • Implements the PID law (continuous and sample form implemented) • Outputs a control signal in the range -1 to 1 which drives the motor voltage
Drive / Motor	• Computes the current motor voltage based on the controller output • Computes the resulting current dynamics using inductance and resistance • Provides the actuation torque acting to the pendulum hinge joint
Pendulum	• Models a 1-DOF mechanical pendulum: rigid massless rod of length L with a point mass m at its end • Takes torque input from the drive, outputs angular position and velocity

(Sima, 2024)

Motivation for Wall Impact Modeling:

- Wall impact **adds hybrid model behavior** (continuous dynamics + discrete events)
- Allows **integration of different modeling approaches** (eq.-based, biomechanical, FEM)
- **Hybrid effect:** Adds non-smooth dynamics (impact, bounce)
- **Introduces discrete events** (zero-crossings at wall position)
- **Physically intuitive and simple** to explain (pendulum hits a wall)
- Extends controlled pendulum **demo from “continuous only” to continuous–discrete hybrid**
- Provides **a natural test case for event handling**
- **Allows higher-fidelity FEM contact models**

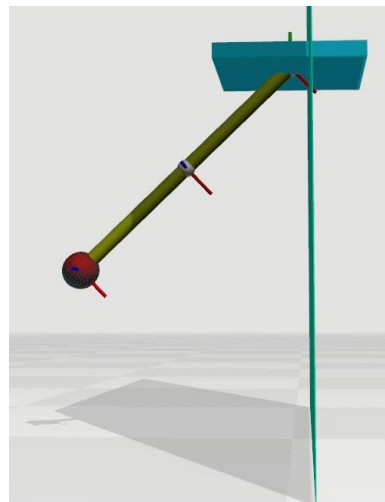
FEM Contact Simulation

- **Static Analysis:**
 - Hertz contact (sphere–plane, hard formulation).
 - Extracts stiffness curve (force vs. penetration).
 - Allows stress/strain fields in contact region
- **Dynamic Analysis:**
 - Implicit Dynamic Simulation
 - Surface interaction: linear-pressure overclosure
- Shows true advantage of FEM integration: **localized stresses and deformations**
- But: limited Python scripting in PrePoMax, integration not trivial
- **Candidates for future:**
 - **NetGen/NGSolve: preprocessing, simulation, and postprocessing within python**
 - CalculiX standalone with Python editing of input decks
 - openCFS (no implicit contact modeling, available via Nitsche Interface Non-Conforming Interface)

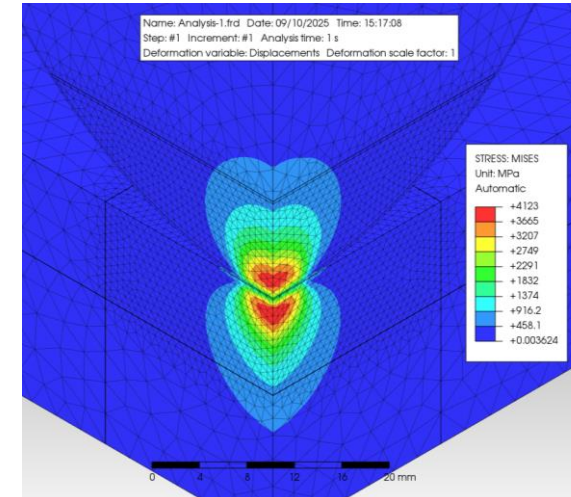
HuntCrossleyForce Definition

Properties	
actions (?)	
name (?)	HuntCrossleyForce
components (?)	(No Objects)
appliesForce (?)	☒
geometry (?)	wall_contact  head_contact  + Add Entry
stiffness	100000.000000 - +
dissipation	0.500000 - +
static_friction	0.900000 - +
dynamic_friction	0.900000 - +
viscous_friction	0.600000 - +
transition_velocity (?)	0.010000 - +

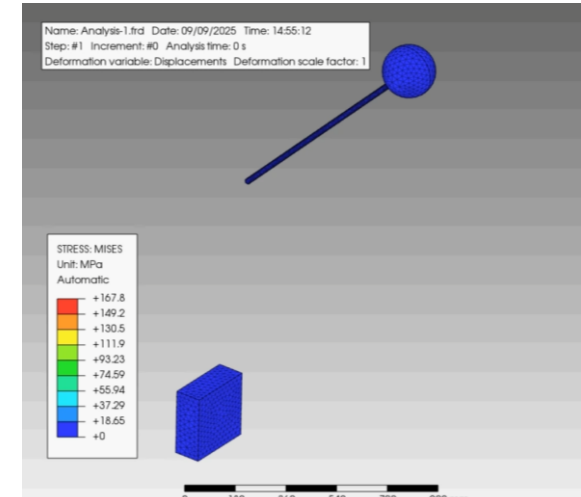
OpenSim Visualization



Static: Hertzian Contact



Dynamic Impact



1. Configuration & Orchestration:

- Implement **YAML-based system configuration** (model locations, parameters, initial states, outputs)
- **Add automatic configuration generation** from FMUs, .osim files, FEM files
 - read metadata → create config templates
- **Build a Configuration Manager:**
 - validate connections, parameters, and unit compatibility
- **Extend the Simulation Master:**
 - Determine execution order (dependency graph)
 - Orchestrate FMUs, OpenSim, FEM in unified loop
 - Manage hybrid events (zero-crossings, reinitializations, discrete jumps).

2. Heterogeneous Modeling:

- **FMU integration:**
 - finalize wrapper (already working with FMPy)
- **OpenSim integration:**
 - implement generic wrapper (parameters, coordinates, actuators, motion export)
- **FEM integration:**
 - Define feasible toolchain (CalculiX wrapper vs. OpenCFS for non-contact problems)
 - Prototype simple FEM subsystem (rod elasticity or Hertz contact)
 - Test co-simulation exchange with SysSimX.

3. Hybrid Behavior & Events

- Extend **interface protocol with event handling** methods
- Demonstrate on demo models:
 - **Wall impact** (continuous & discrete)
 - **Hybrid pendulum with FEM contact**
- Implement test cases for rollback and communication step synchronization.

4. User Interaction & Visualization (planned later)

- Define result handling API:
 - consistent logging
 - metadata storage
- Implement result comparison across configs.
- Add post-processing
- Later: Visual representation of system models (graph layout)